

VOLUME 11 / RUST BACKEND MASTERY

品質保証 — テスト・セキュリティ・可観測性・性能

何を、どの段のテストで確かめるかを決め、ユニットテストから負荷試験・CI までを手を動かして身に付ける

第11巻: 品質保証 — テスト・セキュリティ・可観測性・性能

この巻で身に付けること

第10巻までで、RustNote の画面と機能は、ひととおりそろいました。けれども、「今日、動いた」ことと、「明日も、誰かが直したあとも、ちゃんと動く」ことは別の話です。次のような事故は、動くアプリでも起きます。

- 小さな修正のつもりが、別の画面を壊していたのに、公開するまで誰も気づかなかった
- 英語のデータでは問題が無かったのに、日本語のタイトルを入れたら弾かれた
- 他人のノートが見えてしまう不具合を一度直したのに、半年後に同じ不具合がまた入った
- 公開したら遅くなったが、どこが遅いのか、測る方法が無かった
- 夜中に止まっていたのに、朝、利用者から言われるまで分からなかった

この巻では、こうした事故を、**テスト・見張り（可観測性）・測定**で、早く・自動で見つける方法を学びます。大事なのは、「とにかくテストをたくさん書く」ことではなく、**何を、どの段のテストで確かめるか**を決めることです。

到達目標

この巻を終えると、次のことができるようになります。

- 境界の値（空・1・最大・最大 + 1・日本語・空白）を表に並べた、ユニットテストを書ける。テストが本当に間違いを見つけられるかを、わざと壊した版で確かめられる
- 本物の DB を使った結合テスト、HTTP の返事（状態コード・ヘッダー・本文・クッキー）のテスト、ブラウザで操作する E2E テストを、役割を分けて書ける
- 脅威モデル（第8巻）を、「攻撃が止まること」を確かめる守りのテストに変えられる
- ログ・数の記録（メトリクス）・ヘルスチェックの違いを説明し、速さを release ビルドで測り、負荷をかけて限界を調べられる
- CI で、整形・Clippy・テスト・依存の監査を、毎回自動で回せる

学習地図

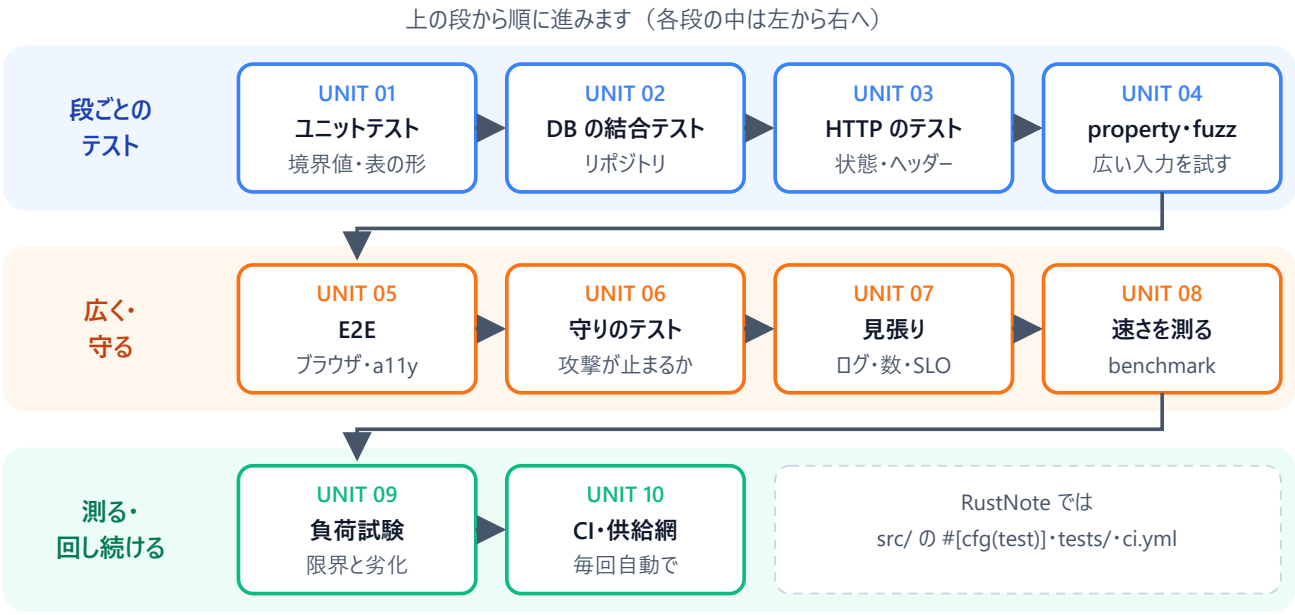


図11-1: 第11巻の学習地図。小さな段のテストから順に積み上げ、広い入力と攻撃を試し、速さと動きを測って、CI で回し続ける

単元	テーマ	ひとことで言うと
UNIT 01	ユニットテスト・境界値・表の形のテスト	端の値を表に並べ、関数1つを速く確かめる
UNIT 02	リポジトリの結合テスト	本物の DB と SQL で、保存と読み出しを確かめる
UNIT 03	HTTP のテスト（準備中）	状態コード・Location・クッキー・本文を確かめる
UNIT 04	property test・fuzz・入力の読み取り（準備中）	思いつかない入力を、たくさん自動で試す
UNIT 05	E2E・ブラウザ・アクセシビリティ（準備中）	大事な流れだけを、本物のブラウザで確かめる
UNIT 06	守りの回帰テスト（準備中）	攻撃が止まることを、毎回確かめる
UNIT 07	tracing・メトリクス・health・SLO（準備中）	動いている様子を外から見えるようにする
UNIT 08	benchmark・profile・メモリの使い方（準備中）	遅いところを、測ってから直す
UNIT 09	負荷試験・背圧・容量（準備中）	たくさん来たときの限界と、壊れ方を決める
UNIT 10	CI・依存・供給網・事故の対応（準備中）	毎回同じ確かめを自動で回し、部品の安全も見る

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03〜10 は準備中で、次の更新から順に追加します。

前提知識（先に読んでおく巻）

- **第2巻（Rust 基礎）**：日本語の文字数を `chars().count()` で数えること（第2巻 UNIT 05）。UNIT 01 の表に、日本語と絵文字の行が出てきます。
- **第4巻（設計）**：検査済みの型（第4巻 UNIT 01）と、テストの書き方の基本（`#[test]`・`assert!`・`assert_eq!`。第4巻 UNIT 10）。
- **第9巻（RustNote 設計）**：RustNote のドメインの型（`NoteTitle`・`Email`・`Password`・`NoteId`）と、その決まり（第9巻 UNIT 03）。この巻のテストも、第9巻と同じ形（`tests/` に足して `cargo test --test 名前`）で書きます。

この巻の進め方

テストは、「通った」だけでは、本当に役に立っているかが分かりません。各単元では、次の順番で進めてください。

1. **決まりを書き出す:** テストを書く前に、確かめる決まり（「タイトルは 1〜80 文字」など）を、日本語で書き出します。
2. **端の値を並べる:** 決まりの両側（80 と 81、0 と 1）と、間違えやすい入力（空白・日本語・絵文字）を表にします。
3. **テストを動かす:** 全部の行が通ることを確かめます。
4. **わざと壊して確かめる:** 関数をわざと間違えた版に変えても、テストがちゃんと失敗するかを確かめます。失敗しないテストは、何も守っていません。

使うツールと版

この巻のコードは、すべて次の環境で実際にコンパイル・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.98 以上**（本書シリーズの基準）。動作確認は rustc 1.99.0 / cargo 1.99.0（`aarch64-pc-windows-msvc`）
RustNote	教材の `Rust/rustnote/`（`rustnote` 0.1.0）。この巻では `src/` を変えずに、`tests/` と `examples/` に確かめ用のファイルを足す
テストの道具	Rust の標準の `#[test]`・`assert!`・`assert_eq!`（UNIT 01 では、外のクレートを足しません）

練習用のフォルダを用意する

第9巻と同じように、教材の `Rust/rustnote/` を**作業用のフォルダにコピー**して使います。

```
# 教材の rustnote フォルダを、中身ごと（-Recurse）作業用のフォルダへコピーする
Copy-Item -Recurse "教材のフォルダ\Rust\rustnote" .\rustnote
cd rustnote
# このフォルダで使われる Rust の版を確かめる（1.99.0 になる）
rustup show active-toolchain
```

UNIT 01: ユニットテスト・境界値・表の形のテスト

この単元のゴール

- テストの「段」（ユニット・結合・HTTP・E2E）の違いを、速さと確かめる範囲で説明できる
- 決まりの両側の値（境界値: 0 と 1、80 と 81）と、間違えやすい入力（空白・全角の空白・日本語・絵文字）を表に並べて、1つのテストで確かめられる
- わざと間違えた版の関数に同じ表を当てて、テストが本当に間違いを見つけられるかを確かめられる
- 文字列の長さの3つの数え方（文字・バイト・UTF-16）の違いを、実際の値で説明できる

たとえ話でつかむ

お菓子の工場で、袋詰めの機械を検査する場面を考えます。決まりは「1袋に 1〜80 個」です。

- 検査の人は、40 個のような**ちょうど真ん中**の袋だけを試すことはしません。真ん中が正しくても、端で間違える機械はよくあるからです。**0 個・1 個・80 個・81 個**の4つを必ず試します。決まりの**境目の両側**です。
- 形が崩れたお菓子・小さなお菓子・2つがくっついたお菓子のように、**数え方を間違えやすいもの**も試します。
- 検査の結果は、**表**に書きます。「何を入れたか・どうなるはずか・どうなったか」を1行ずつ並べれば、どこで間違えたかが一目で分かります。
- そして、検査の表そのものが正しいかも確かめます。わざと**壊した機械**に同じ検査をして、ちゃんと「不合格」が出るかを見るのです。壊れた機械まで合格にしてしまう検査は、意味がありません。

お菓子の工場	RustNote のテスト
「1袋に 1〜80 個」の決まり	`NoteTitle::parse` の「前後の空白を除いて 1〜80 文字」
0・1・80・81 個の袋	**境界値**（決まりの端と、その外側）
数え方を間違えやすいお菓子	全角の空白・日本語・絵文字・濁点を分けて書いた「が」
検査の表	**表の形のテスト**（入力と期待する結果を、1行ずつ並べたテスト）
壊した機械で検査を試す	**わざと間違えた版**で、テストが失敗するかを見る

> **用語:** ユニットテスト（unit test：関数や型のような小さな部品1つを、ほかから切り離して確かめるテスト。DB もネットワークも使わないので速い）

> **用語:** 境界値（きょうかいち：決まりが切り替わる境目の値。「1〜80 文字」なら、0・1・80・81 文字）

> **用語:** 表の形のテスト（ひょうのかたちのテスト。table-driven test：入力と期待する結果の組を表（配列）に並べ、同じ確かめ方で1行ずつ試すテスト）

図で理解する

テストには、確かめる範囲の違う「段」があります。RustNote にもともと入っている 41 件のテストを、段ごとに分けると次のようになります。

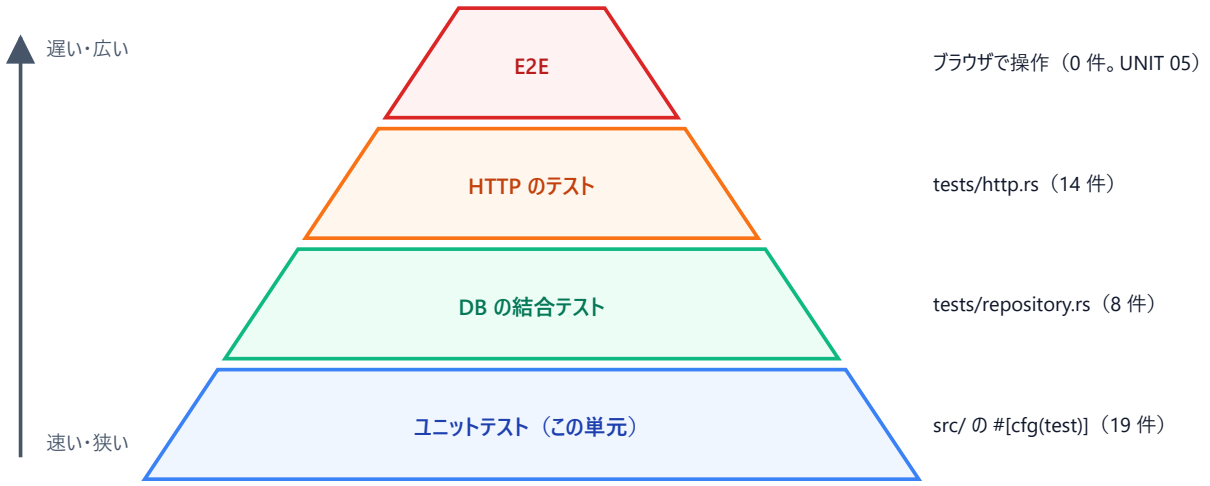


図11-2: テストの段。下の段ほど速く、確かめる範囲が狭いので、たくさん書ける。上の段ほど遅く、広い範囲を一度に確かめるので、大事な流れだけに絞る。右は RustNote にもともと入っているテストの件数

- **ユニットテスト**は、関数1つを確かめます。DB もネットワークも使わないので、数百件あっても1秒もかかりません。決まりの細かい端（80 文字と 81 文字など）は、ここで確かめます。
- 上の段になるほど、本物の DB や HTTP の流れまで含めて確かめられますが、遅く、失敗したときに原因の場所が分かりにくくなります。だから、**細かいことは下の段で、つながりは上の段で**、と役割を分けます。

境界値は、次の図のように選びます。

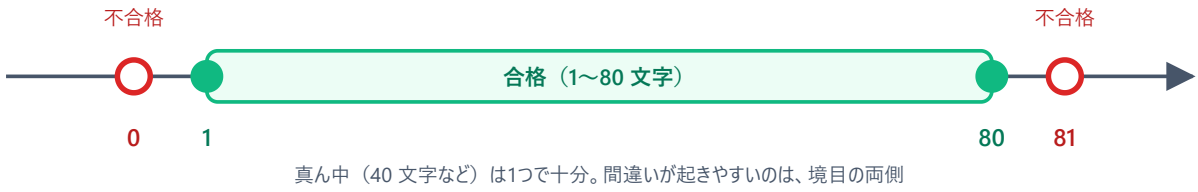


図11-3:「1〜80 文字」の境界値。決まりの端（1 と 80）と、その1つ外側（0 と 81）を必ず試す

- `>` と `>=` の書き間違い、`len()` と `chars().count()` の取り違えのような間違いは、真ん中の値では見つかりません。**端と、その1つ外側**で初めて、答えが変わるからです。

まず動かす最小コード

例1-1: NoteTitle::parse を、表の形で確かめる

> ▼ **このコードがやること（先に日本語で）**：タイトルの決まりを確かめる行（名前・入力・期待する結果）を、表（`Vec<Case>`）に並べます。表のすべての行を `NoteTitle::parse` に渡し、期待どおりなら `ok`、違えば `NG` と、実際の結果を表示します。合わなかった行が1つでもあれば、テストを失敗させます。同じ表を、わざと間違えた版の関数（文字数を `len()` で数える）にも当てて、表が間違いを見つけられるかも確かめます。

```rust
//! 第11巻 UNIT 01: NoteTitle::parse を、境界の値を並べた表で確かめる（表の形のテスト）。
//! 実行: cargo test --test u01_title_table -- --nocapture

use rustnote::domain::NoteTitle;

/// 期待する結果: 合格して入る値か、不合格のときのエラーの文
enum Expect {
    Ok(String),
    Err(&'static str),
}

/// 表の1行: 名前・入力・期待する結果
struct Case {
    label: &'static str,
    input: String,
    expect: Expect,
}

fn ok(label: &'static str, input: impl Into<String>, value: impl Into<String>) -> Case
{
    Case {
        label,
        input: input.into(),
        expect: Expect::Ok(value.into()),
    }
}

fn err(label: &'static str, input: impl Into<String>, message: &'static str) -> Case {
    Case {
        label,
        input: input.into(),
        expect: Expect::Err(message),
    }
}

/// タイトルの決まり（前後の空白を除いて 1〜80 文字）を、境界の値で並べた表
fn title_cases() -> Vec<Case> {
    let a80 = "あ".repeat(80);
    vec![
        ok("ふつうの日本語", "牛乳を買う", "牛乳を買う"),
        ok("前後の空白は取る", "  買い物  ", "買い物"),
        err("空", "", "タイトルを入力してください"),
        err("半角の空白だけ", "   ", "タイトルを入力してください"),
        err("全角の空白だけ", "\u{3000}", "タイトルを入力してください"),
        ok("1文字（下の端）", "あ", "あ"),
        ok("80文字（上の端）", a80.clone(), a80.clone()),
        err(
            "81文字（上の端 + 1）",
            "あ".repeat(81),
```

```
            "タイトルは80文字以内にしてください（現在81文字）",
        ),
        ok("空白を除けば80文字", format!(" {a80} "), a80),
        ok(
            "絵文字の家族（見た目は1つ）",
            "👨\u{200D}👩\u{200D}👧",
            "👨\u{200D}👩\u{200D}👧",
        ),
    ]
}

/// 表のすべての行を `parse` で試し、合わなかった行の説明を返す
fn check(parse: impl Fn(&str) -> Result<String, String>) -> Vec<String> {
    let mut failures = Vec::new();
    for case in title_cases() {
        let actual = parse(&case.input);
        let matched = match (&case.expect, &actual) {
            (Expect::Ok(value), Ok(got)) => value == got,
            (Expect::Err(message), Err(got)) => message == got,
            _ => false,
        };
        let mark = if matched { "ok  " } else { "NG  " };
        let shown = match &actual {
            Ok(got) if got.chars().count() > 10 => {
                format!("Ok（{} 文字）", got.chars().count())
            }
            Ok(got) => format!("Ok({got:?})"),
            Err(message) => format!("Err({message})"),
        };
        println!("{mark}{} → {shown}", case.label);
        if !matched {
            failures.push(case.label.to_string());
        }
    }
    failures
}

/// RustNote の NoteTitle::parse を、表の形にそろえて呼ぶ
fn rustnote_parse(input: &str) -> Result<String, String> {
    NoteTitle::parse(input)
        .map(|title| title.as_str().to_string())
        .map_err(|error| error.to_string())
}

#[test]
fn note_title_follows_the_table() {
    let failures = check(rustnote_parse);
    assert!(failures.is_empty(), "表と合わなかった行: {failures:?}");
}
```

```rust
/// わざと間違えた版: 文字数を len()（バイト数）で数えている
fn broken_parse(input: &str) -> Result<String, String> {
    let trimmed = input.trim();
    let len = trimmed.len();
    if len == 0 {
        return Err("タイトルを入力してください".to_string());
    }
    if len > 80 {
        return Err(format!(
            "タイトルは80文字以内にしてください（現在{len}文字）"
        ));
    }
    Ok(trimmed.to_string())
}

#[test]
fn the_table_catches_a_broken_version() {
    let failures = check(broken_parse);
    println!("わざと間違えた版で合わなかった行: {failures:?}");
    // 表が、間違いを見つけられること（合わない行が1つ以上あること）を確かめる
    assert!(!failures.is_empty(), "表が間違いを見つけられませんでした");
}
```

コピーした `rustnote` フォルダの `tests/u01_title_table.rs` として保存して実行します。

```
cargo test --test u01_title_table -- --nocapture --test-threads=1
```

▼ **実行結果**（最初の1回は `Compiling` の行が続きます。ここでは省いています）

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 32.43s
     Running tests\u01_title_table.rs (target\debug\deps\u01_title_table-
6a49fd9e5e4e501b.exe)

running 2 tests
test note_title_follows_the_table ... ok  ふつうの日本語 → Ok("牛乳を買う")
ok  前後の空白は取る → Ok("買い物")
ok  空 → Err(タイトルを入力してください)
ok  半角の空白だけ → Err(タイトルを入力してください)
ok  全角の空白だけ → Err(タイトルを入力してください)
ok  1文字（下の端） → Ok("あ")
ok  80文字（上の端） → Ok（80 文字）
ok  81文字（上の端 + 1） → Err(タイトルは80文字以内にしてください（現在81文字）)
ok  空白を除けば80文字 → Ok（80 文字）
ok  絵文字の家族（見た目は1つ） → Ok("👨\u{200d}👩\u{200d}👧")
ok
test the_table_catches_a_broken_version ... ok  ふつうの日本語 → Ok("牛乳を買う")
ok  前後の空白は取る → Ok("買い物")
ok  空 → Err(タイトルを入力してください)
ok  半角の空白だけ → Err(タイトルを入力してください)
ok  全角の空白だけ → Err(タイトルを入力してください)
ok  1文字（下の端） → Ok("あ")
NG  80文字（上の端） → Err(タイトルは80文字以内にしてください（現在240文字）)
NG  81文字（上の端 + 1） → Err(タイトルは80文字以内にしてください（現在243文字）)
NG  空白を除けば80文字 → Err(タイトルは80文字以内にしてください（現在240文字）)
ok  絵文字の家族（見た目は1つ） → Ok("👨\u{200d}👩\u{200d}👧")
わざと間違えた版で合わなかった行: ["80文字（上の端）", "81文字（上の端 + 1）", "空白を除けば80文
字"]
ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

▼ コードを1つずつ分解して解説

解説1: 表の1行に、名前を付ける

- `Case` は、表の1行です。`label`（行の名前）・`input`（入力）・`expect`（期待する結果）の3つを持ちます。`ok(...)` と `err(...)` は、行を短く書くための小さな関数です。
- 行の名前に「80文字（上の端）」「81文字（上の端 + 1）」のように、**なぜその行があるのか**を書いておきます。失敗したときに、どの決まりの、どの端で間違えたのかが、名前だけで分かります（エラー1）。
- `"\u{3000}"` は全角の空白、`"\u{200D}"` は、絵文字どうしをつなぐ「見えない文字」（ゼロ幅接合子）です。見えない文字は、ソースに直接書くと区別できないので、`\u{...}` の形で書きます。

解説2: 1つの確かめ方を、全部の行に当てる

- `check` は、表のすべての行に、同じ確かめ方を当てる関数です。比べたい関数を引数（`parse`）で受け取るので、RustNote の `NoteTitle::parse` にも、わざと間違えた版にも、同じ表を使えます。
- 1行ずつ `assert!` で止めずに、合わなかった行の名前を `failures` に集めてから、最後にまとめて `assert!` しています。こうすると、1回の実行で、**合わない行を全部**見つけられます。

解説3: RustNote の版は、全部の行が ok

- 1つ目のテスト（`note_title_follows_the_table`）では、10 行すべてが `ok` でした。半角の空白だけ・全角の空白だけは「タイトルを入力してください」、81 文字は「タイトルは80文字以内にしてください（現在81文字）」です。前後に空白を付けた 82 文字の入力は、空白を除くと 80 文字なので合格でした。
- 家族の絵文字の行は、`Ok("👨\u{200d}👩\u{200d}👧")` と表示されました。`{:?}` で表示すると、見えない文字（ゼロ幅接合子）が `\u{200d}` と書き出されます。

解説4: 間違えた版は、80 と 81 の行で見つかった

- 2つ目のテスト（`the_table_catches_a_broken_version`）では、文字数を `len()`（バイト数）で数える版に、同じ表を当てました。`NG` になったのは、80 文字・81 文字・空白を除けば 80 文字の3行です。日本語の「あ」は1文字3バイトなので、80 文字は 240 バイトと数えられ、「長すぎる」と断られました。
- 一方、「牛乳を買う」（15 バイト）や、家族の絵文字（18 バイト）の行は、間違えた版でも `ok` でした。**短い日本語だけの表では、この間違いを見つけられない**のです。境界値の行を入れておいたから、見つけられました。
- このテストは、「間違えた版で、合わない行が1つ以上あること」を `assert!` で確かめています。表を直していて、うっかり境界値の行を消してしまったら、このテストが失敗して教えてくれます。

もう一歩: 実務に近い形

例1-2: 「長さ」の3つの数え方を比べる

タイトルの文字数は、`chars().count()` で数えています。でも、人が見て「1文字」と思うものと、プログラムの数え方は、いつも同じではありません。

> **▼ このコードがやること（先に日本語で）**: いくつかの文字列について、`chars().count()`（Unicode の文字の数）・`len()`（UTF-8 のバイト数）・`encode_utf16().count()`（UTF-16 の単位の数）の3つを表示し、`NoteTitle::parse` が合格にするかも表示します。

```rust
//! 1つの文字列の「長さ」を、3通りの数え方で比べる。
//! NoteTitle::parse は chars().count()（Unicode の文字の数）で数えている。
//! 実行: cargo run -q --example u01_counts

use rustnote::domain::NoteTitle;

fn main() {
    let samples = [
        ("A", "A"),
        ("あ", "あ"),
        ("が（1文字で表した形）", "\u{304C}"),
        ("が（か + 濁点の2つで表した形）", "\u{304B}\u{3099}"),
        ("👍", "👍"),
        ("家族の絵文字", "👨\u{200D}👩\u{200D}👧"),
        ("ゼロ幅スペースだけ", "\u{200B}"),
    ];
    for (label, text) in samples {
        let parsed = match NoteTitle::parse(text) {
            Ok(_) => "合格".to_string(),
            Err(error) => format!("不合格（{error}）"),
        };
        println!("[{label}]");
        println!(
            "    文字 {} ／ バイト {} ／ UTF-16 {} ／ NoteTitle::parse は {parsed}",
            text.chars().count(),
            text.len(),
            text.encode_utf16().count()
        );
    }
}
```

```
cargo run -q --example u01_counts
```

▼ 実行結果

```
[A]
    文字 1 ／ バイト 1 ／ UTF-16 1 ／ NoteTitle::parse は 合格
[あ]
    文字 1 ／ バイト 3 ／ UTF-16 1 ／ NoteTitle::parse は 合格
[が（1文字で表した形）]
    文字 1 ／ バイト 3 ／ UTF-16 1 ／ NoteTitle::parse は 合格
[が（か + 濁点の2つで表した形）]
    文字 2 ／ バイト 6 ／ UTF-16 2 ／ NoteTitle::parse は 合格
[👍]
    文字 1 ／ バイト 4 ／ UTF-16 2 ／ NoteTitle::parse は 合格
[家族の絵文字]
    文字 5 ／ バイト 18 ／ UTF-16 8 ／ NoteTitle::parse は 合格
[ゼロ幅スペースだけ]
    文字 1 ／ バイト 3 ／ UTF-16 1 ／ NoteTitle::parse は 合格
```



図11-4: 1つに見える家族の絵文字の、3つの数え方。RustNote のタイトルの長さは「文字」で数えるので、この絵文字は 5 文字になる

- **「が」は、2通りの書き方がある:** 1文字で表した「が」（U+304C）は 1 文字ですが、「か」と濁点（U+3099）を組み合わせた「が」は 2 文字です。見た目は同じでも、数は違います。入力の方法（コピーした元の文書など）によって、どちらが来るかは変わります。
- **家族の絵文字は、5 文字:** 見た目は1つですが、👴・つなぐ文字・👵・つなぐ文字・👧の5文字でできています。RustNote では、この絵文字を 16 個入れると、見た目は 16 個でも 80 文字になり、17 個目で断られます。
- **ゼロ幅スペースだけでも合格する:** 見えない文字1つのタイトルは、`NoteTitle::parse` を通ってしまいます。第9巻 UNIT 03 で見つけた、RustNote に残っている改善点です（コードは変えていません）。
- **UTF-16 の数との食い違い:** ブラウザの入力欄の `maxlength`（入力できる長さの上限）は、HTML の決まりでは UTF-16 の単位で数えます。RustNote の作成のフォームは `maxlength=(MAX_TITLE_CHARS)`（80）を付けているので、決まりどおりに数えるブラウザでは、👍 のように UTF-16 で 2 単位の絵文字だけのタイトルは、40 個までしか入りません。サーバーは 80 文字（80 個）まで受け付けるので、ブラウザのほうが先に止めることになります。実際のブラウザでの数え方は、ブラウザを使ったテストで確かめます（UNIT 05（準備中））。どの数え方で「長さ」を決めるかは、仕様として決めて、表のテストに書いておきます。

> **注意:** 人が見て「1文字」と思う単位（書記素クラスタ）で数えるには、標準ライブラリではなく、`unicode-segmentation` のような外のクレートが必要です。RustNote は、SQLite の `length()` と同じ数え方（Unicode の文字の数）にそろえることを優先して、`chars().count()` を使っています（第2巻 UNIT 05・第9巻 UNIT 03）。

よくあるエラーと読み方

この節で作るテストのファイルは、確かめ終わったら必ず消してください。`tests/` に残しておくと、`cargo test` のたびに失敗したり、コンパイルが止まったりします。

エラー1: 表の1行が、期待と合わない

表の「期待する答え」を、まちがえて書いた場合です（`tests/e01_wrong_row.rs`）。

```
use rustnote::domain::NoteTitle;

#[test]
fn title_length_rows() {
    // 表の2行目の「期待する答え」を、まちがえて true（合格）にしてしまった
    let cases = [
        ("80文字", "あ".repeat(80), true),
        ("81文字", "あ".repeat(81), true),
    ];
    for (label, input, should_pass) in cases {
        assert_eq!(
            NoteTitle::parse(&input).is_ok(),
            should_pass,
            "{label} の行が合いません"
        );
    }
}
```

```
cargo test --test e01_wrong_row
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.22s
     Running tests\e01_wrong_row.rs (target\debug\deps\e01_wrong_row-
41a050fd370554c8.exe)

running 1 test
test title_length_rows ... FAILED

failures:

---- title_length_rows stdout ----

thread 'title_length_rows' (151252) panicked at tests\e01_wrong_row.rs:11:9:
assertion `left == right` failed: 81文字 の行が合いません
  left: false
 right: true
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    title_length_rows

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--test e01_wrong_row`
```

- **何が起きたか:** `assertion left == right failed: 81文字 の行が合いません` は、「`assert_eq!` の左（実際の結果）と右（期待）が違いました。81文字 の行です」という意味です。`left: false` は、`NoteTitle::parse` が不合格（`is_ok()` が `false`）を返したこと、`right: true` は、表に「合格のはず」と書いたことです。
- **どちらが間違いか:** テストが失敗したとき、間違っているのは関数のほうとは限りません。今回は、81 文字を合格と書いた**表のほう**が、決まり（1〜80 文字）と違っていました（第9巻 UNIT 01）。
- **名前を付ける理由:** `assert_eq!` の3つ目の引数に `{label}` を入れたので、表のどの行で失敗したかが、すぐ分かりました。名前が無いと、`left: false` と `right: true` だけでは、何の行か分かりません。

エラー2: async fn に #[test] を付けた

DB を使うテストを書くときに、async の関数に、ふつうの `#[test]` を付けてしまった場合です（`tests/e02_async.rs`）。

```rust
use rustnote::domain::NoteTitle;

// async fn に、ふつうの #[test] を付けてしまった
#[test]
async fn title_is_checked() {
    assert!(NoteTitle::parse("買い物").is_ok());
}
```

```
cargo test --test e02_async
```

```
   Compiling rustnote v0.1.0 (...\rustnote)
error: async functions cannot be used for tests
 --> tests\e02_async.rs:5:1
  |
5 |   async fn title_is_checked() {
  |   ^----
  |   |
  |  _`async` because of this
  | |
6 | |     assert!(NoteTitle::parse("買い物").is_ok());
7 | | }
  | |_^

error: could not compile `rustnote` (test "e02_async") due to 1 previous error
```

- **何が起きたか:** 「async の関数は、テストに使えません」というエラーです。ふつうの `#[test]` は、関数を呼ぶだけなので、async の関数の中身（`.await` で待つ部分）を動かせません。
- **直し方:** async の中身が要るテストには、`#[tokio::test]` を付けます。Tokio のランタイムを用意してから、関数を動かしてくれます（第5巻 UNIT 02・第9巻 UNIT 01）。この例の `NoteTitle::parse` は async ではないので、`async` を消して、ふつうの `fn` にするのが正しい直し方です。

エラー3（エラーが出ない間違い）：呼ぶだけで、確かめていないテスト

わざと間違えた版の関数を呼んでいるのに、結果を確かめていないテストです（`tests/e03_no_assert.rs`）。

```rust
/// わざと間違えた版: 文字数を len()（バイト数）で数えている
fn broken_parse(input: &str) -> Result<String, String> {
    let len = input.trim().len();
    if len == 0 || len > 80 {
        return Err("不合格".to_string());
    }
    Ok(input.trim().to_string())
}

#[test]
fn eighty_characters() {
    // 呼ぶだけで、結果を確かめていない
    let _ = broken_parse(&"あ".repeat(80));
}
```

```
cargo test --test e03_no_assert
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 1.16s
     Running tests\e03_no_assert.rs (target\debug\deps\e03_no_assert-
5a6ef5e20346be07.exe)

running 1 test
test eighty_characters ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

- **何が起きたか:** 間違えた版（80 文字の日本語を「長すぎる」と断る）なのに、テストは `ok` でした。`let _ = ...` で結果を捨てているので、何が返ってきても失敗しません。テストは、**panic したら失敗、しなければ成功**という仕組みだからです。
- **見つけ方:** 例1-1のように、わざと間違えた版を当てて、テストが失敗するかを確かめます。失敗しないテストは、件数を増やすだけで、何も守っていません。この「わざと間違いを入れて、テストが気づくか」を道具で自動にしたものを、ミューテーションテストと呼びます（UNIT 04（準備中））。

RustNote ではここで使う

RustNote の `src/domain.rs` の最後には、`#[cfg(test)]` のユニットテストが入っています。タイトルの境界値のテストは、次の形です（抜き出しています）。

```
    #[test]
    fn note_title_counts_characters_not_bytes() {
        // 日本語 80 文字は 240 バイトだが、文字数では 80 なので合格
        let eighty = "あ".repeat(80);
        assert!(NoteTitle::parse(&eighty).is_ok());

        let eighty_one = "あ".repeat(81);
        assert_eq!(
            NoteTitle::parse(&eighty_one),
            Err(TitleError::TooLong { len: 81, max: 80 })
        );
    }
```

- `#[cfg(test)]` の付いた `mod tests` は、`cargo test` のときだけコンパイルされます。本番の実行ファイルには入りません。
- 80 文字と 81 文字の境界を、**日本語で**確かめています。例1-1の「間違えた版」で見たとおり、英数字だけで試していたら、`len()` の取り違えに気づけません。
- 81 文字のときは、`is_err()` で「失敗した」ことだけでなく、`Err(TitleError::TooLong { len: 81, max: 80 })` と、**どう失敗したか**まで比べています。違う理由で失敗している（たとえば「空です」と言われている）間違いも見つけられます。

RustNote の 19 件のユニットテストは、次のコマンドで一覧にできます。`--lib` は「ライブラリの中の `#[cfg(test)]` のテストだけ」という指定です。

```
cargo test --lib -- --list
```

練習問題

問1: `Email::parse` の決まり（前後の空白を除いて小文字にそろえる・`@` の前後が空でない・ドメインに `.` がある・空白が無い・254 バイトまで）を、例1-1と同じ表の形のテストにしてください。境界値として、254 バイトと 255 バイトの行も入れます。

問2: URL の `/notes/{id}` の `{id}` は、`"42".parse::<NoteId>()` で `NoteId` にします。`"1"`・`"0"`・`"-1"`・`"9223372036854775807"`（`i64` の最大値）・`"9223372036854775808"`（最大値 + 1）・`"abc"`・`" 1"`（前に空白）・`"01"` を試して、それぞれどうなるかを表示してください。結果で気になる点はありますか。

問3: `Password::parse` の境界（8〜128 文字）を確かめてください。7・8・128・129 文字のほかに、「空白 7 つ + `a`」と「日本語 8 文字」の行も入れます。パスワードの空白は、タイトルと同じように取り除かれるでしょうか。

解答と解説

問1の解答:

`tests/u01_q1.rs` です。例1-1の `Case` を使わずに、タプル（名前, 入力, 期待）の `Vec` で、もっと短く書いています。

```rust
//! 問1の解答: Email::parse を、境界と「よくある書き間違い」を並べた表で確かめる。
//! 実行: cargo test --test u01_q1 -- --nocapture

use rustnote::domain::Email;

#[test]
fn email_follows_the_table() {
    let long_ok = format!("{}@example.com", "a".repeat(242));
    let long_ng = format!("{}@example.com", "a".repeat(243));
    //  (名前, 入力, 期待: Ok なら そろえた値、Err なら エラーの文)
    let cases: Vec<(&str, String, Result<String, &str>)> = vec![
        (
            "ふつう",
            "alice@example.com".into(),
            Ok("alice@example.com".into()),
        ),
        (
            "空白と大文字",
            "  Alice@Example.COM  ".into(),
            Ok("alice@example.com".into()),
        ),
        (
            "@ が無い",
            "alice".into(),
            Err("メールアドレスに @ がありません"),
        ),
        (
            "@ の前が空",
            "@example.com".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "@ の後ろが空",
            "alice@".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "ドメインに . が無い",
            "alice@example".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "ドメインが . で始まる",
            "alice@.example.com".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "途中に空白",
            "al ice@example.com".into(),
```

```
            Err("メールアドレスの形式が正しくありません"),
        ),
        (
            "@ が2つ",
            "a@b@example.com".into(),
            Err("メールアドレスの形式が正しくありません"),
        ),
        ("254 バイト（上の端）", long_ok.clone(), Ok(long_ok)),
        (
            "255 バイト（上の端 + 1）",
            long_ng,
            Err("メールアドレスが長すぎます"),
        ),
    ];
    let mut failures = Vec::new();
    for (label, input, expected) in cases {
        let actual = Email::parse(&input)
            .map(|email| email.as_str().to_string())
            .map_err(|error| error.to_string());
        let matched = match (&expected, &actual) {
            (Ok(value), Ok(got)) => value == got,
            (Err(message), Err(got)) => message == got,
            _ => false,
        };
        let shown = match &actual {
            Ok(got) if got.len() > 30 => format!("Ok（{} バイト）", got.len()),
            Ok(got) => format!("Ok({got:?})"),
            Err(message) => format!("Err({message})"),
        };
        println!("{} {label} → {shown}", if matched { "ok" } else { "NG" });
        if !matched {
            failures.push(label);
        }
    }
    assert!(failures.is_empty(), "表と合わなかった行: {failures:?}");
}
```

```
cargo test --test u01_q1 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.46s
     Running tests\u01_q1.rs (target\debug\deps\u01_q1-81fa10c5a5eabbd3.exe)

running 1 test
ok ふつう → Ok("alice@example.com")
ok 空白と大文字 → Ok("alice@example.com")
ok @ が無い → Err(メールアドレスに @ がありません)
ok @ の前が空 → Err(メールアドレスの形式が正しくありません)
ok @ の後ろが空 → Err(メールアドレスの形式が正しくありません)
ok ドメインに . が無い → Err(メールアドレスの形式が正しくありません)
ok ドメインが . で始まる → Err(メールアドレスの形式が正しくありません)
ok 途中に空白 → Err(メールアドレスの形式が正しくありません)
ok @ が2つ → Err(メールアドレスの形式が正しくありません)
ok 254 バイト（上の端） → Ok（254 バイト）
ok 255 バイト（上の端 + 1） → Err(メールアドレスが長すぎます)
test email_follows_the_table ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

- 11 行すべてが通りました。`" Alice@Example.COM "` は、空白が取られ、小文字の `alice@example.com` にそろいました（第9巻 UNIT 03）。
- 254 バイトの行は、`a` を 242 個 + `@example.com`（12 バイト）で作っています。上限はバイト数（`normalized.len()`）で決めているので、境界値もバイト数で作ります。決まりがどの数え方かによって、境界値の作り方も変わります。
- `"a@b@example.com"` は、最初の `@` で分けると、ドメインが `b@example.com` になり、その中に `@` があるので断られました。

問2の解答:

`tests/u01_q2.rs` です。

```rust
//! 問2の解答: URL の /notes/{id} の {id} を NoteId にするときの境界を確かめる。
//! 実行: cargo test --test u01_q2 -- --nocapture

use rustnote::domain::NoteId;

#[test]
fn note_id_boundaries() {
    let inputs = [
        "1",
        "0",
        "-1",
        "9223372036854775807",
        "9223372036854775808",
        "abc",
        " 1",
        "01",
    ];
    for input in inputs {
        match input.parse::<NoteId>() {
            Ok(id) => println!("{input:?} → Ok({})", id.get()),
            Err(error) => println!("{input:?} → Err({error})"),
        }
    }
    // 下の端と上の端を、assert で固定する
    assert!("0".parse::<NoteId>().is_err());
    assert!("1".parse::<NoteId>().is_ok());
    assert!("9223372036854775807".parse::<NoteId>().is_ok());
    assert!("9223372036854775808".parse::<NoteId>().is_err());
}
```

```
cargo test --test u01_q2 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.65s
     Running tests\u01_q2.rs (target\debug\deps\u01_q2-d924c1589f8ead12.exe)

running 1 test
"1" → Ok(1)
"0" → Err(ID は 1 以上の数です)
"-1" → Err(ID は 1 以上の数です)
"9223372036854775807" → Ok(9223372036854775807)
"9223372036854775808" → Err(ID は数字で指定してください)
"abc" → Err(ID は数字で指定してください)
" 1" → Err(ID は数字で指定してください)
"01" → Ok(1)
test note_id_boundaries ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.02s
```

- 下の端は、`"1"` が合格、`"0"` と `"-1"` が「ID は 1 以上の数です」でした。上の端は、`i64` の最大値 `9223372036854775807` が合格、1 つ大きい `9223372036854775808` が不合格です。
- **気になる点1:** `9223372036854775808` は数なのに、「ID は数字で指定してください」と言われました。`NoteId` は、まず `i64` として読み、読めなければ「数字ではない」としているからです（`src/domain.rs` の `from_str`）。大きすぎる数も「数字ではない」になります。URL の ID は利用者が手で打つものではないので大きな問題ではありませんが、エラーの文が正確でないことは、テストを書いて初めて分かります。
- **気になる点2:** `"01"` は `1` として合格しました。つまり `/notes/01` と `/notes/1` は、同じノートを開きます。困ることはありませんが、「同じものを指す URL が2つある」ことは、知っておくと、ログを調べるときに迷いません。
- `" 1"`（前に空白）は不合格でした。`i64` の `parse` は、空白を取り除かないからです。タイトルの `trim()` と比べると、型ごとに空白の扱いが違うことが分かります。

問3の解答:

`tests/u01_q3.rs` です。

```rust
//! 問3の解答: Password::parse の境界（8〜128 文字）と、空白の扱いを確かめる。
//! 実行: cargo test --test u01_q3 -- --nocapture

use rustnote::domain::Password;

#[test]
fn password_boundaries() {
    let cases = [
        ("7文字（下の端 − 1）", "a".repeat(7), false),
        ("8文字（下の端）", "a".repeat(8), true),
        ("128文字（上の端）", "a".repeat(128), true),
        ("129文字（上の端 + 1）", "a".repeat(129), false),
        ("空白7つ + a（8文字）", format!("{}a", " ".repeat(7)), true),
        ("日本語8文字（24 バイト）", "あ".repeat(8), true),
    ];
    for (label, input, should_pass) in cases {
        let result = Password::parse(&input);
        match &result {
            Ok(password) => println!("{label} → Ok({password:?})"),
            Err(error) => println!("{label} → Err({error})"),
        }
        assert_eq!(result.is_ok(), should_pass, "{label} の行が合いません");
    }
}
```

```
cargo test --test u01_q3 -- --nocapture
```

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 3.84s
     Running tests\u01_q3.rs (target\debug\deps\u01_q3-f9715f282720c475.exe)

running 1 test
7文字（下の端 − 1） → Err(パスワードは8文字以上にしてください)
8文字（下の端） → Ok(Password(***))
128文字（上の端） → Ok(Password(***))
129文字（上の端 + 1） → Err(パスワードは128文字以内にしてください)
空白7つ + a（8文字） → Ok(Password(***))
日本語8文字（24 バイト） → Ok(Password(***))
test password_boundaries ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.00s
```

- 7 文字は「パスワードは8文字以上にしてください」、129 文字は「パスワードは128文字以内にしてください」、8 文字と 128 文字は合格です。合格の行は `Password(***)` と表示され、中身は出ません（第9巻 UNIT 03）。

- 「空白 7 つ + `a`」も 8 文字として合格しました。パスワードは、タイトルと違って `trim()` しません。`src/domain.rs` のコメントに「パスワードは空白も含めて利用者が決めたものなので、trim しない」とあります。空白をこっそり取ると、利用者が決めたパスワードと違うものが保存されてしまうからです。
- 「日本語 8 文字」（24 バイト）も合格でした。パスワードの長さも `chars().count()`（文字の数）で数えています。

まとめ

- テストには段がある。ユニットテストは、関数1つを速く確かめる段で、決まりの細かい端はここで確かめる。上の段（DB・HTTP・E2E）は、つながりを確かめる段で、大事な流れに絞る。
- 境界値（端と、その1つ外側）と、間違えやすい入力（空白・全角の空白・日本語・絵文字）を、名前付きの行で表に並べる。1つの確かめ方を全部の行に当て、合わない行をまとめて報告する。
- わざと間違えた版に同じ表を当てて、テストが失敗することを確かめる。結果を確かめないテストは、何も守っていない。

UNIT 02: リポジトリの結合テスト

この単元のゴール

- リポジトリ（`NoteRepository` など）を、偽物ではなく**本物の DB**（一時フォルダの libSQL ファイル）で確かめる理由を、SQL の細かい決まりの例で説明できる
- 準備（Arrange）・操作（Act）・確かめ（Assert）の形で、成功・見つからない・他人・版の食い違いを、1つずつ確かめるテストを書ける
- わざと SQL を壊したとき、ユニットテストは通ったまま、結合テストだけが気づく様子を確かめられる
- テストどうしが DB ファイルを使い回したときに起きる、実行のたびに変わる失敗を読み、テストごとに新しい DB を作る形に直せる

たとえ話でつかむ

レストランの厨房で、新しいスープの味を確かめる場面を考えます。

- UNIT 01 のユニットテストは、**材料1つずつの味見**です。「塩はこの量」「だしはこの濃さ」と、1つずつ確かめます。速くて、どこが悪いかもすぐ分かります。
- ところが、材料が全部正しくても、**実際の鍋で煮る**と、火加減や、入れる順番のせいで、思った味にならないことがあります。そこで、本物の鍋で煮て味見をします。これが、この単元の**結合テスト**です。
- 味見のたびに、**洗った新しい鍋**を使います。前の人が使った鍋のままだと、前のスープの味が残っていて、どちらの味なのか分からなくなります。2人が同時に1つの鍋を使えば、もっと混乱します。

厨房	RustNote のテスト
材料1つずつの味見	ユニットテスト（`escape_like` や `NoteTitle::parse` を、DB なしで確かめる。UNIT 01）
本物の鍋で煮て味見	リポジトリの結合テスト（本物の libSQL に、本物の SQL を流す）
火加減・入れる順番	SQL の細かい決まり（`LIKE` の `ESCAPE`・`WHERE` の条件・`UNIQUE`・`ON DELETE CASCADE`）
洗った新しい鍋	テストごとに作る一時フォルダの DB（`common::test_db()`）
前のスープの味が残る	前のテストのデータが残った DB。結果が実行のたびに変わる

> **用語:** 結合テスト（けつごうテスト：いくつかの部品を、本物どうしでつないで確かめるテスト。ここでは、リポジトリと本物の DB をつなぐ）

> **用語:** Arrange・Act・Assert（アレンジ・アクト・アサート：テストを「準備する」「操作する」「確かめる」の3つに分けて書く形）

図で理解する

テストの段と、それぞれが確かめるものです。この単元は、下から2段目です。

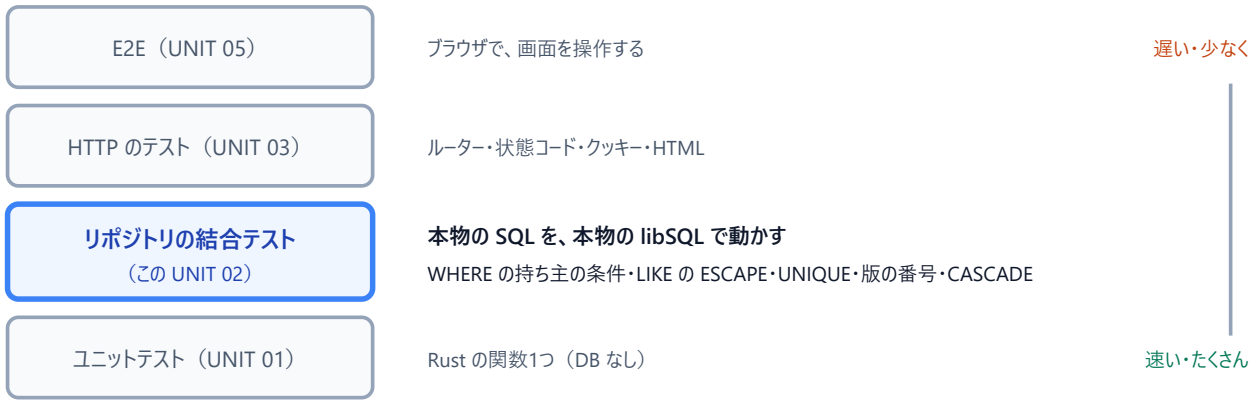


図11-5: テストの段。リポジトリの結合テストは、ユニットテストでは確かめられない「SQL そのもの」を、本物の DB で確かめる

テストごとに、新しい DB を用意します。



図11-6: テストごとの DB。RustNote の common::test_db() は、テストごとに一時フォルダを作り、マイグレーションを当てた空の DB を渡す

まず動かす最小コード

例2-1: NoteRepository の約束を、1つずつ確かめる

`NoteRepository` の6つのメソッドには、それぞれ約束があります（`src/repository/notes.rs` のコメント）。たとえば「他人のノート・存在しないノートは None」「`expected_version` が今の version と違えば `VersionMismatch`」です。これを、本物の DB で1つずつ確かめます。

> **▼ このコードがやること（先に日本語で）**：空の DB に alice と bob を登録します（準備）。alice のノートを作り、探す・更新する・完了にする・消す、を、alice と bob の両方から試します（操作）。それぞれの結果を、1行ずつ表示します（確かめ）。ここでは結果を見るために表示していますが、本物のテストでは `assert!` で確かめます。

```rust
//! 第11巻 UNIT 02: NoteRepository の約束を、本物の DB（一時フォルダの libSQL ファイル）で1つずつ確かめる。
//! 実行: cargo test --test u02_contract -- --nocapture

mod common;

use rustnote::{
    domain::{Email, NewNote, Note, NoteId},
    error::RepoError,
    repository::{LibsqlNoteRepository, LibsqlUserRepository, NoteRepository, UserRepository},
};

/// ノートの大事な所だけを、1行の文字列にする
fn brief(result: Result<Note, RepoError>) -> String {
    match result {
        Ok(note) => format!(
            "Ok (id={} version={} done={} title={:?}) ",
            note.id, note.version, note.done, note.title
        ),
        Err(error) => format!("Err({error:?})"),
    }
}

fn new_note(title: &str) -> NewNote {
    NewNote::parse(title, "").expect("テスト用のノート")
}

#[tokio::test]
async fn note_repository_contract() {
    // Arrange: 空の DB と、2人のユーザー
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let notes = LibsqlNoteRepository::new(test.db.clone());
    let alice = users
        .create(
            &Email::parse("alice@example.com").unwrap(),
            "$argon2id$dummy",
        )
        .await
        .unwrap();
    let bob = users
        .create(&Email::parse("bob@example.com").unwrap(), "$argon2id$dummy")
        .await
        .unwrap();

    // Act と Assert: 約束を1つずつ
    let created = notes.insert(alice.id, &new_note("初版")).await.unwrap();
    let id = created.id;
```

```
    println!(
        "insert (alice)              → {}",
        brief(Ok(created.clone()))
    );
    println!(
        "   作成と更新の日時が同じ    → {}",
        created.created_at == created.updated_at
    );

    let found = notes.find(alice.id, id).await.unwrap();
    println!(
        "find (alice)                → 見つかる: {}",
        found.is_some()
    );
    let found = notes.find(bob.id, id).await.unwrap();
    println!(
        "find (bob)                  → 見つかる: {}",
        found.is_some()
    );

    let result = notes.update(alice.id, id, &new_note("第2版"), 1).await;
    println!("update (alice, version 1)   → {}", brief(result));
    let result = notes.update(alice.id, id, &new_note("古い画面"), 1).await;
    println!("update (alice, version 1)   → {}", brief(result));
    let missing = NoteId::new(99).unwrap();
    let result = notes.update(alice.id, missing, &new_note("無い"), 1).await;
    println!("update (alice, id 99)       → {}", brief(result));
    let result = notes.update(bob.id, id, &new_note("乗っ取り"), 2).await;
    println!("update (bob, version 2)     → {}", brief(result));

    let result = notes.toggle_done(alice.id, id).await;
    println!("toggle_done (alice)         → {}", brief(result));

    println!(
        "delete (bob)                → {:?}",
        notes.delete(bob.id, id).await
    );
    println!(
        "delete (alice)              → {:?}",
        notes.delete(alice.id, id).await
    );
    println!(
        "delete (alice, もう一度)    → {:?}",
        notes.delete(alice.id, id).await
    );
}
```

コピーした `rustnote` フォルダの `tests/u02_contract.rs` として保存して実行します。

```
cargo test --test u02_contract -- --nocapture
```

▼ 実行結果

```
    Finished `test` profile [unoptimized + debuginfo] target(s) in 6.44s
     Running tests\u02_contract.rs (target\debug\deps\u02_contract-
8e9944192195f42d.exe)

running 1 test
insert (alice)              → Ok (id=1 version=1 done=false title="初版")
   作成と更新の日時が同じ    → true
find (alice)                → 見つかる: true
find (bob)                  → 見つかる: false
update (alice, version 1)   → Ok (id=1 version=2 done=false title="第2版")
update (alice, version 1)   → Err(VersionMismatch)
update (alice, id 99)       → Err(NotFound)
update (bob, version 2)     → Err(NotFound)
toggle_done (alice)         → Ok (id=1 version=3 done=true title="第2版")
delete (bob)                → Err(NotFound)
delete (alice)              → Ok(())
delete (alice, もう一度)    → Err(NotFound)
test note_repository_contract ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.09s
```

▼ コードを1つずつ分解して解説

解説1: 準備は、毎回まっさらな DB から

- `common::test_db().await` が、新しい一時フォルダに DB を作り、マイグレーション（表を作る SQL。第13巻 UNIT 02）を当てて渡します。だから、最初のノートの id は、いつも 1 です。
- ユーザーは `LibsqlUserRepository::create` で登録します。パスワードのハッシュは、ここでは関係ないので `"$argon2id$dummy"` という仮の文字列です。

解説2: 成功だけでなく、断られる場合を全部試す

- `update` は4回呼びました。正しい version（1）なら成功して version 2 に、同じ version 1 でもう一度なら `VersionMismatch`（版の食い違い。別のタブで先に更新された）、無い id 99 なら `NotFound`、bob からなら `NotFound` です。
- bob からの更新は、version を正しい 2 にしても `NotFound` でした。「他人のノートは、あっても無いのと同じ」という約束です（第8巻 UNIT 07）。`VersionMismatch` にすると、「そのノートはある」と bob に教えてしまいます。
- 削除は、bob から → `NotFound`、alice から → `Ok(())`、もう一度 → `NotFound` です。1回目と3回目の `NotFound` は、理由は違います（他人／もう無い）が、外から見ると同じです。

解説3: 本物の DB でも、テストは速い

- 実行結果の最後の行のとおり、このテストは数十ミリ秒で終わります。一時フォルダのファイルなので、ネットワークの待ち時間がありません。RustNote の `tests/repository.rs` の8つのテストも、まとめて 1 秒かかりません。
- 速いので、「本物の DB は遅いから偽物にする」必要は、ここではありません。

もう一歩: 実務に近い形

例2-2: SQL を1か所壊すと、どのテストが気づくか

本物の DB で確かめると、何が良いのでしょうか。SQL をわざと1か所壊して、確かめます。試すときは、練習用のコピーの `src/repository/notes.rs` を書き換え、確かめたら必ず元に戻してください。

検索の SQL の、`ESCAPE '\'`（`\` のうしろの文字を、ただの文字として扱う、という指定）を消します。

```
        // 元: "user_id = ?1 AND (title LIKE ?2 ESCAPE '\\' OR body LIKE ?2 ESCAPE
'\\')"
        let condition = "user_id = ?1 AND (title LIKE ?2 OR body LIKE ?2)";
```

まず、`escape_like` のユニットテスト（`src/repository/notes.rs` の中の `like_special_characters_are_escaped`）を動かします。

```
cargo test --lib like_special
```

```
running 1 test
test repository::notes::tests::like_special_characters_are_escaped ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 18 filtered out; finished
in 0.00s
```

ユニットテストは**通りました**。次に、リポジトリの結合テストを動かします。

```
cargo test --test repository
```

```
running 8 tests
test migrations_are_applied_only_once ... ok
test duplicate_email_is_a_unique_violation ... ok
test deleting_a_session_logs_out ... ok
test sessions_expire ... ok
test update_uses_optimistic_locking ... ok
test notes_are_isolated_per_owner ... ok
test toggle_and_delete ... ok
test search_filters_by_keyword_and_paginates ... FAILED

failures:

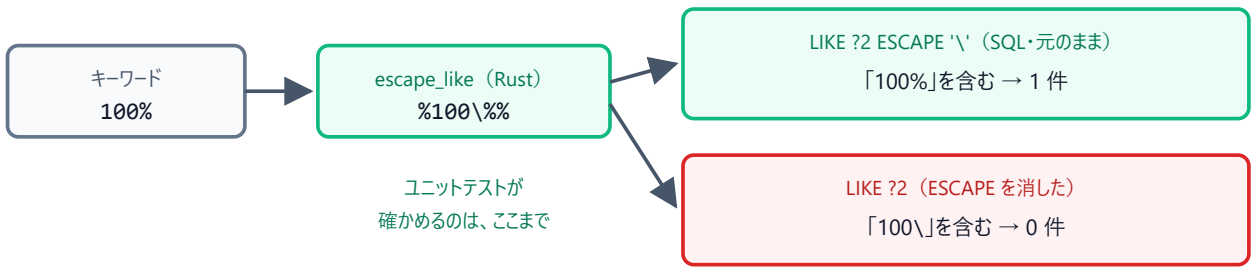
---- search_filters_by_keyword_and_paginates stdout ----

thread 'search_filters_by_keyword_and_paginates' (198152) panicked at
tests\repository.rs:198:5:
assertion `left == right` failed
  left: 0
 right: 1
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    search_filters_by_keyword_and_paginates

test result: FAILED. 7 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.25s

error: test failed, to rerun pass `--test repository`
```



作った模様（Rust）が正しくても、SQL がその模様をどう読むかは、本物の DB でしか確かめられない

図11-7: ESCAPE を消したときの違い。escape_like の結果は同じでも、SQL が \ を「ただの \」として読むので、何も見つからなくなる

- **何が起きたか:** 8つのテストのうち、`search_filters_by_keyword_and_paginates` だけが失敗しました。198行目は `assert_eq!(percent.total_count, 1);` で、「100%」で検索した件数が、1 のはずが 0 でした。

- **なぜ 0 件か:** `escape_like` は、`100%` を `100\%` に書き換えます（これは正しいままです）。ところが SQL に `ESCAPE '\'` が無いと、`\` は「次の文字をただの文字にする印」ではなく、**ただの** `\` として読まれます。だから「`100\` を含むノート」を探してしまい、1件も見つかりません。
- **学べること:** Rust の関数（`escape_like`）と、SQL（`ESCAPE '\'`）は、**2つそろって**初めて正しく動きます。ユニットテストは片方しか見ていないので、もう片方が壊れても気づけません。偽物のリポジトリ（第4巻 UNIT 02）でも、SQL は動かないので気づけません。**SQL を書いたら、本物の DB で動かすテストを置く**のが、この単元の中心です。確かめたら、`src/repository/notes.rs` を元に戻します。

よくあるエラーと読み方

エラー1: `assertion failed: matches!(…)` だけで、何が返ったか分からない

今度は、削除の SQL から、持ち主の条件 `AND user_id = ?2` を消してみます（練習用のコピーで試し、必ず元に戻します）。

```
                "DELETE FROM notes WHERE id = ?1",
```

```
cargo test --test repository
```

```
running 8 tests
test migrations_are_applied_only_once ... ok
test duplicate_email_is_a_unique_violation ... ok
test deleting_a_session_logs_out ... ok
test sessions_expire ... ok
test update_uses_optimistic_locking ... ok
test toggle_and_delete ... ok
test notes_are_isolated_per_owner ... FAILED
test search_filters_by_keyword_and_paginates ... ok

failures:

---- notes_are_isolated_per_owner stdout ----

thread 'notes_are_isolated_per_owner' (187916) panicked at tests\repository.rs:90:5:
assertion failed: matches!(notes.delete(bob.id, note.id).await,
Err(RepoError::NotFound))
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    notes_are_isolated_per_owner

test result: FAILED. 7 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.22s

error: test failed, to rerun pass `--test repository`
```

- **何が起きたか:** `notes_are_isolated_per_owner` が、90 行目で失敗しました。bob が alice のノートを消そうとしたら `NotFound` になるはずが、そうならなかった、という意味です。テストが無ければ、**誰でも、他人のノートを id だけで消せる**ところでした（IDOR。第8巻 UNIT 01）。
- **気づきにくい点:** SQL の `?2` を消したのに、`params![id, owner]` で2つの値を渡したままでも、エラーにはなりませんでした。libsql は、使われない値を黙って無視しました。**コンパイルも、実行も通る**ので、テストだけが頼りです。
- **困る点:** `assertion failed: matches!(…)` は、**実際に何が返ったか**を表示しません。`Ok(())` だったのか、別のエラーだったのか、この表示だけでは分かりません。

直し方: `assert!` の2つ目の引数に、失敗したときの文と、実際の値を書いておきます（`tests/u02_owner_delete.rs`）。

```rust
//! 第11巻 UNIT 02 のエラー1の直し方: 失敗したときに「実際の値」が出る形で確かめる。
//! 実行: cargo test --test u02_owner_delete

mod common;

use rustnote::{
    domain::{Email, NewNote},
    error::RepoError,
    repository::{LibsqlNoteRepository, LibsqlUserRepository, NoteRepository,
UserRepository},
};

#[tokio::test]
async fn other_user_cannot_delete() {
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let notes = LibsqlNoteRepository::new(test.db.clone());
    let alice = users
        .create(
            &Email::parse("alice@example.com").unwrap(),
            "$argon2id$dummy",
        )
        .await
        .unwrap();
    let bob = users
        .create(&Email::parse("bob@example.com").unwrap(), "$argon2id$dummy")
        .await
        .unwrap();
    let note = notes
        .insert(alice.id, &NewNote::parse("買い物", "").unwrap())
        .await
        .unwrap();

    let result = notes.delete(bob.id, note.id).await;
    // 2つ目の引数に、失敗したときに出す文と、実際の値を書いておく
    assert!(
        matches!(result, Err(RepoError::NotFound)),
        "bob の削除は NotFound のはずが、{result:?} だった"
    );
    let still_there = notes.find(alice.id, note.id).await.unwrap();
    assert!(still_there.is_some(), "alice のノートが消えている");
}
```

壊したままの SQL で動かすと、次のように表示されます。

```
cargo test --test u02_owner_delete
```

```
running 1 test
test other_user_cannot_delete ... FAILED

failures:

---- other_user_cannot_delete stdout ----

thread 'other_user_cannot_delete' (196796) panicked at tests\u02_owner_delete.rs:35:5:
bob の削除は NotFound のはずが、Ok(()) だった
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    other_user_cannot_delete

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.05s

error: test failed, to rerun pass `--test u02_owner_delete`
```

- 「bob の削除は NotFound のはずが、`Ok(())` だった」と、実際の値が出ました。「bob の削除が**成功してしまった**」と、すぐに分かります。
- 元の SQL に戻すと、このテストは通ります（`test other_user_cannot_delete ... ok`）。**失敗したときに読む人**のために、確かめる所に一言書いておくのが、良いテストです。

エラー2: テストが、同じ DB ファイルを使い回した

テストごとに DB を作るのが面倒で、決まった1つのファイル（`rustnote-shared-test.db`）を、どのテストからも開くようにした場合です（`tests/u02_shared.rs`）。2つのテストは、どちらも alice を登録します。

```rust
//! 第11巻 UNIT 02 のエラー2: 2つのテストが、同じ DB ファイルを使い回したとき。
//! 実行: cargo test --test u02_shared

use rustnote::{
    db::{self, Db},
    domain::Email,
    repository::{LibsqlUserRepository, UserRepository},
};

/// どのテストも、決まった1つのファイルを開いてしまう（一時フォルダを作らない）
async fn shared_db() -> Db {
    let path = std::env::temp_dir().join("rustnote-shared-test.db");
    let db = Db::open_local(path).await.expect("DB を開けません");
    db::migrate(&db).await.expect("マイグレーションに失敗");
    db
}

#[tokio::test]
async fn sign_up_alice() {
    let users = LibsqlUserRepository::new(shared_db().await);
    let email = Email::parse("alice@example.com").unwrap();
    users.create(&email, "$argon2id$dummy").await.unwrap();
}

#[tokio::test]
async fn alice_can_be_found() {
    let users = LibsqlUserRepository::new(shared_db().await);
    let email = Email::parse("alice@example.com").unwrap();
    users.create(&email, "$argon2id$dummy").await.unwrap();
    assert!(users.find_by_email(&email).await.unwrap().is_some());
}
```

ファイルが無い状態から、1回目を動かします。

```
cargo test --test u02_shared
```

```
running 2 tests
test sign_up_alice ... ok
test alice_can_be_found ... FAILED

failures:

---- alice_can_be_found stdout ----

thread 'alice_can_be_found' (194980) panicked at tests\u02_shared.rs:29:51:
called `Result::unwrap()` on an `Err` value: UniqueViolation("SQLite failure: `UNIQUE
constraint failed: users.email`")
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace


failures:
    alice_can_be_found

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.04s

error: test failed, to rerun pass `--test u02_shared`
```

- **何が起きたか:** `sign_up_alice` は通り、`alice_can_be_found` が `UniqueViolation`（同じメールアドレスが、もうある）で失敗しました。2つのテストは**同時に**動くので、先に alice を登録したほうが通り、あとのほうが失敗します。どちらが先かは、その時しだいです。

同じことを、ファイルを消してから 8 回くり返すと、毎回1つは通り、もう1つの失敗の理由は、次のように変わりました。

失敗の理由	回数（8 回のうち）	どこで止まったか
`UniqueViolation`（alice がもういる）	6 回	alice を登録するところ
`database is locked`（ほかの接続が書いている最中）	1 回	DB を開くところ
`table notes already exists`（表がもうある）	1 回	マイグレーションのところ

```
thread 'sign_up_alice' (186904) panicked at tests\u02_shared.rs:13:41:
DB を開けません: SqliteFailure(5, "database is locked")
thread 'alice_can_be_found' (197884) panicked at tests\u02_shared.rs:14:28:
マイグレーションに失敗: Apply { version: "0002_notes", source: SqliteFailure(1, "table
notes already exists") }
```

- 2つのテストが、同じファイルを**同時に**開いて、同時にマイグレーションを当てようとするので、「表を作る」「書き込む」がぶつかります。どこでぶつかるかは、毎回違います。

ファイルを消さずに、もう一度動かします。

```
running 2 tests
test sign_up_alice ... FAILED
test alice_can_be_found ... FAILED

failures:

---- sign_up_alice stdout ----

thread 'sign_up_alice' (198816) panicked at tests\u02_shared.rs:22:51:
called `Result::unwrap()` on an `Err` value: UniqueViolation("SQLite failure: `UNIQUE
constraint failed: users.email`")
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

---- alice_can_be_found stdout ----

thread 'alice_can_be_found' (185664) panicked at tests\u02_shared.rs:29:51:
called `Result::unwrap()` on an `Err` value: UniqueViolation("SQLite failure: `UNIQUE
constraint failed: users.email`")


failures:
    alice_can_be_found
    sign_up_alice

test result: FAILED. 0 passed; 2 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.02s

error: test failed, to rerun pass `--test u02_shared`
```

- 今度は、**両方とも**失敗しました。前の実行で登録した alice が、ファイルに残っているからです。
- **直し方**: RustNote のテストのように、`common::test_db()` で、テストごとに新しい一時フォルダの DB を作ります。テストの結果が「そのテストの中でしたこと」だけで決まるようにすると、同時に動かしても、何回動かしても、同じ結果になります。確かめたら、`tests/u02_shared.rs` と、一時フォルダ（PowerShell なら `$env:TEMP`）にできた `rustnote-shared-test.db` で始まるファイルを消しておきます。
- **学べること**: 「ときどき失敗するテスト」は、テストどうしが何かを**共有している**ことが多い原因です。DB のファイル・決まったポート番号・環境変数・今の時刻などです。失敗したら、まず「このテストは、ほかのテストと何を共有しているか」を探します。

RustNote ではここで使う

テストごとの DB は、`tests/common/mod.rs` の `test_db` が作ります（抜き出しています）。

```rust
/// 一時フォルダの DB。TempDir が消えるとフォルダごと削除される
pub struct TestDb {
    pub db: Db,
    _dir: TempDir,
}

/// マイグレーション済みの空の DB を作る
pub async fn test_db() -> TestDb {
    let dir = tempfile::tempdir().expect("一時フォルダを作れません");
    let db = Db::open_local(dir.path().join("test.db"))
        .await
        .expect("DB を開けません");
    db::migrate(&db).await.expect("マイグレーションに失敗");
    TestDb { db, _dir: dir }
}
```

- `tempfile::tempdir()` は、ほかと重ならない名前の一時フォルダを作ります。テストが同時にいくつ動いても、それぞれ別のフォルダです。
- `TestDb` は、`db` と一緒に `TempDir` を持っています。テストが終わって `TestDb` が片付けられる（drop される）と、`TempDir` もフォルダごと消えます（第3巻 UNIT 01）。項目の名前が `_dir` と `_` で始まるのは、「使わないが、持っておくこと自体に意味がある」という印です。`TempDir` を持っておくのは、テストの間はフォルダを残し、終わったら消すためです。
- 本番と同じ `db::migrate` を当てるので、テストの DB の表は、いつも本番と同じ形です。マイグレーションを足せば、テストの DB にも自動で入ります。

`tests/repository.rs` は、この `test_db` を使って、リポジトリの約束を8つのテストで確かめています。ユーザーを登録する手順は、何度も出てくるので、小さな関数にまとめています（抜き出しています）。

```rust
/// テスト用のユーザーを登録する
async fn create_user(users: &LibsqlUserRepository, email: &str) -> User {
    let email = Email::parse(email).unwrap();
    users.create(&email, "$argon2id$dummy").await.unwrap()
}
```

```
cargo test --test repository
```

```
running 8 tests
test migrations_are_applied_only_once ... ok
test duplicate_email_is_a_unique_violation ... ok
test update_uses_optimistic_locking ... ok
test deleting_a_session_logs_out ... ok
test sessions_expire ... ok
test notes_are_isolated_per_owner ... ok
test toggle_and_delete ... ok
test search_filters_by_keyword_and_paginates ... ok

test result: ok. 8 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.21s
```

- 8つのテストは、同時に動いて、0.2 秒ほどで終わりました。どれも自分の DB を持っているので、同時に動いてもぶつかりません。

練習問題

問1: タイトル `Rust 入門`・本文 `SQLite の LIKE` のノートと、タイトル `Äpfel kaufen` のノートを作り、`Rust`・`rust`・`RUST`・`ｒｕｓｔ`（全角）・`sqlite`・`Äpfel`・`äpfel` で検索して、それぞれ何件見つかるかを表示してください。検索は、大文字と小文字を区別するでしょうか。

問2: セッションの期限を、決まった時刻 T にして作り、`find_user` を「T の1秒前」「T ちょうど」「T の1秒後」で呼んでください。`delete_expired` も「T の1秒前」と「T ちょうど」で呼び、何件消えたかを表示してください。期限ちょうどの時刻は、まだ使えるでしょうか。

問3: alice と bob に、ノートを2件ずつとセッションを1つずつ作ってから、SQL で alice を `users` 表から消し、`users`・`notes`・`sessions` の行の数が、消す前と後でどう変わるかを表示してください。

解答と解説

問1の解答:

`tests/u02_q1.rs` です。

```rust
//! 問1の解答: 検索（LIKE）は、大文字と小文字を区別するかを確かめる。
//! 実行: cargo test --test u02_q1 -- --nocapture

mod common;

use rustnote::{
    domain::{Email, NewNote, NoteQuery},
    repository::{LibsqlNoteRepository, LibsqlUserRepository, NoteRepository,
UserRepository},
};

#[tokio::test]
async fn like_and_letter_case() {
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let notes = LibsqlNoteRepository::new(test.db.clone());
    let alice = users
        .create(
            &Email::parse("alice@example.com").unwrap(),
            "$argon2id$dummy",
        )
        .await
        .unwrap();
    for (title, body) in [("Rust 入門", "SQLite の LIKE"), ("Äpfel kaufen", "")] {
        notes
            .insert(alice.id, &NewNote::parse(title, body).unwrap())
            .await
            .unwrap();
    }

    for keyword in [
        "Rust",
        "rust",
        "RUST",
        "ｒｕｓｔ",
        "sqlite",
        "Äpfel",
        "äpfel",
    ] {
        let page = notes
            .search(alice.id, &NoteQuery::new(Some(keyword), None))
            .await
            .unwrap();
        println!("{keyword:<8} → {} 件", page.total_count);
    }
}
```

```
cargo test --test u02_q1 -- --nocapture
```

```
running 1 test
Rust     → 1 件
rust     → 1 件
RUST     → 1 件
ｒｕｓｔ     → 0 件
sqlite   → 1 件
Äpfel    → 1 件
äpfel    → 0 件
test like_and_letter_case ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.05s
```

- `Rust`・`rust`・`RUST` は、どれも 1 件でした。SQLite の `LIKE` は、**英字（ASCII）の大文字と小文字を区別しません。**本文にだけある `sqlite` も見つかりました（タイトルと本文の両方を探す SQL だからです）。
- ところが、全角の `ｒｕｓｔ` は 0 件、`äpfel` も 0 件でした。SQLite の `LIKE` が大文字と小文字を同じに扱うのは、ASCII の英字だけで、`Ä` と `ä` のような文字は、別の文字として比べます。全角と半角も、別の文字です。
- これは、RustNote の仕様として決めておくことです。今の RustNote は「英字は大文字と小文字を区別しない。それ以外は、書いたとおりに探す」です。仕様を変えるなら、このテストの表を先に書き換えてから、SQL を直します。

問2の解答:

`tests/u02_q2.rs` です。

```rust
//! 問2の解答: セッションの期限ちょうどの時刻で、探す・掃除するとどうなるかを確かめる。
//! 実行: cargo test --test u02_q2 -- --nocapture

mod common;

use std::time::{Duration, SystemTime, UNIX_EPOCH};

use rustnote::{
    domain::Email,
    repository::{
        LibsqlSessionRepository, LibsqlUserRepository, SessionRepository,
UserRepository,
    },
};
use topcoat::session::TokenHash;

#[tokio::test]
async fn session_expiry_boundary() {
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let sessions = LibsqlSessionRepository::new(test.db.clone());
    let alice = users
        .create(
            &Email::parse("alice@example.com").unwrap(),
            "$argon2id$dummy",
        )
        .await
        .unwrap();

    // 期限（T）を、決まった時刻にしておく（今の時刻に頼らない）
    let t = UNIX_EPOCH + Duration::from_secs(2_000_000_000);
    let second = Duration::from_secs(1);
    let hash = TokenHash::new([7; 32]);
    sessions.create(&hash, alice.id, t).await.unwrap();

    for (label, now) in [
        ("T − 1秒", t - second),
        ("T ちょうど", t),
        ("T + 1秒", t + second),
    ] {
        let found = sessions.find_user(&hash, now).await.unwrap();
        println!("find_user（{label}）→ 見つかる: {}", found.is_some());
    }
    let before: SystemTime = t - second;
    println!(
        "delete_expired（T − 1秒）→ {} 件消した",
        sessions.delete_expired(before).await.unwrap()
    );
    println!(
```

```
        "delete_expired（T ちょうど）→ {} 件消した",
        sessions.delete_expired(t).await.unwrap()
    );
}
```

```
cargo test --test u02_q2 -- --nocapture
```

```
running 1 test
find_user（T − 1秒）→ 見つかる: true
find_user（T ちょうど）→ 見つかる: false
find_user（T + 1秒）→ 見つかる: false
delete_expired（T − 1秒）→ 0 件消した
delete_expired（T ちょうど）→ 1 件消した
test session_expiry_boundary ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.07s
```

- 期限ちょうどの T では、もう見つかりませんでした。`find_user` の SQL は `s.expires_at > ?2`（期限が今より**あと**なら有効）で、`delete_expired` は `expires_at <= ?1`（期限が今か、今より**前**なら消す）です。T ちょうどは「使えない」側と「消す」側の両方に入るので、すき間も重なりもありません。
- 境界の「ちょうど」は、UNIT 01 の境界値と同じで、いちばん間違えやすい所です。`>` と `>=` を取り違えると、「期限ちょうどの1秒だけ、使えないのに消されない」セッションができます。
- 時刻を `SystemTime::now()` ではなく、決まった時刻（`UNIX_EPOCH` から 20 億秒）にしたのは、テストの結果を、動かした時刻によって変えないためです（エラー2の「共有しているもの」の1つが、今の時刻です）。

問3の解答:

`tests/u02_q3.rs` です。

```rust
//! 問3の解答: ユーザーを消すと、その人のノートとセッションはどうなるかを確かめる。
//! 実行: cargo test --test u02_q3 -- --nocapture

mod common;

use std::time::{Duration, SystemTime};

use rustnote::{
    db::Db,
    domain::{Email, NewNote},
    repository::{
        LibsqlNoteRepository, LibsqlSessionRepository, LibsqlUserRepository,
NoteRepository,
        SessionRepository, UserRepository,
    },
};
use topcoat::session::TokenHash;

/// 表ごとの行の数を、1行の文字列にする
async fn counts(db: &Db) -> String {
    let conn = db.connect().await.expect("DB に接続できません");
    let mut parts = Vec::new();
    for table in ["users", "notes", "sessions"] {
        let mut rows = conn
            .query(&format!("SELECT COUNT(*) FROM {table}"), ())
            .await
            .expect("数えられません");
        let row = rows
            .next()
            .await
            .expect("行を読めません")
            .expect("行がない");
        let count: i64 = row.get(0).expect("COUNT(*)");
        parts.push(format!("{table}={count}"));
    }
    parts.join(" ")
}

#[tokio::test]
async fn deleting_a_user_cascades() {
    let test = common::test_db().await;
    let users = LibsqlUserRepository::new(test.db.clone());
    let notes = LibsqlNoteRepository::new(test.db.clone());
    let sessions = LibsqlSessionRepository::new(test.db.clone());
    let expires = SystemTime::now() + Duration::from_secs(60);
    for (n, email) in ["alice@example.com", "bob@example.com"]
        .into_iter()
        .enumerate()
    {
```

```
        let user = users
            .create(&Email::parse(email).unwrap(), "$argon2id$dummy")
            .await
            .unwrap();
        for title in ["1件目", "2件目"] {
            notes
                .insert(user.id, &NewNote::parse(title, "").unwrap())
                .await
                .unwrap();
        }
        let hash = TokenHash::new([n as u8; 32]);
        sessions.create(&hash, user.id, expires).await.unwrap();
    }
    println!("消す前: {}", counts(&test.db).await);

    // RustNote には「退会」の機能がまだ無いので、SQL で直接 alice を消す
    let conn = test.db.connect().await.unwrap();
    let deleted = conn
        .execute("DELETE FROM users WHERE email = 'alice@example.com'", ())
        .await
        .unwrap();
    println!("users から {deleted} 行消した");
    println!("消した後: {}", counts(&test.db).await);
}
```

```
cargo test --test u02_q3 -- --nocapture
```

```
running 1 test
消す前: users=2 notes=4 sessions=2
users から 1 行消した
消した後: users=1 notes=2 sessions=1
test deleting_a_user_cascades ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in
0.08s
```

- `users` から alice の1行を消すと、alice のノート2件とセッション1つも、一緒に消えました。bob の分は残っています。
- `migrations` の表の定義で、`notes.user_id` と `sessions.user_id` に `REFERENCES users(id) ON DELETE CASCADE`（参照先が消えたら、一緒に消す）と書いてあり、libSQL の接続では外部キーの検査が有効（`foreign_keys=1`。第13巻 UNIT 02）だからです。
- もし外部キーの検査が無効だったら、持ち主のいないノートが残ります。こうした**表の約束**も、SQL と同じで、本物の DB でしか確かめられません。RustNote に「退会」の機能を作るときは、このテストを、その機能のテストに育てます。

まとめ

- リポジトリは、本物の DB（一時フォルダの libSQL ファイル）で確かめる。Rust の関数が正しくても、SQL の `ESCAPE` や `WHERE` の持ち主の条件が壊れていれば、ユニットテストは通ったままになる。
- 準備・操作・確かめの形で、成功だけでなく、見つからない・他人・版の食い違いを1つずつ確かめる。`assert!` には、失敗したときの文と実際の値を書いておく。
- テストごとに新しい DB を作り、テストどうしで何も共有しない。共有すると、結果が実行のたびに変わり、前の実行のデータにも引きずられる。

この巻のまとめ

この版に収録した UNIT 01〜02 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: ユニットテスト・境界値・表の形のテスト

- ☐ テストの段（ユニット・DB の結合・HTTP・E2E）の違いと、細かい決まりを下の段で確かめる理由を説明できる
- ☐「1〜80 文字」のような決まりから、境界値（0・1・80・81）と、間違えやすい入力（空白・全角の空白・日本語・絵文字）を選べる
- ☐ 名前付きの行を表に並べ、1つの確かめ方を全部の行に当てて、合わない行をまとめて報告するテストを書ける
- ☐ わざと間違えた版に同じ表を当てて、テストが失敗することを確かめられる。短い日本語だけの表では `len()` の間違いを見つけられない理由を言える
- ☐ 文字（`chars().count()`）・バイト（`len()`）・UTF-16 の単位の3つの数え方の違いを、絵文字と「が」で説明できる
- ☐ `assert_eq!` の失敗の表示（`left` と `right`）を読み、表と関数のどちらが間違っているかを考えられる
- ☐ `#[test]` を async fn に付けたエラーと、結果を確かめないテストが何も守っていないことを説明できる

UNIT 02: リポジトリの結合テスト

- ☐ リポジトリを、偽物ではなく本物の DB（一時フォルダの libSQL ファイル）で確かめる理由を、`LIKE` の `ESCAPE` の例で説明できる
- ☐ 準備・操作・確かめの形で、成功・見つからない・他人・版の食い違いを、1つずつ確かめるテストを書ける
- ☐ SQL から `ESCAPE` を消すと、ユニットテストは通ったまま、結合テストだけが失敗することを確かめられる
- ☐ 削除の SQL から持ち主の条件を消したときの失敗を読み、`assert!` に実際の値を出す文を足せる
- ☐ 1つの DB ファイルを使い回したテストが、実行のたびに違う理由（UNIQUE・locked・表がもうある）で失敗することと、`common::test_db()` で直す理由を言える
- ☐ 検索の大文字と小文字・セッションの期限ちょうど・ユーザーを消したときの CASCADE を、本物の DB で確かめられる

> **お知らせ:** この版には UNIT 01〜02 を収録しています。UNIT 03（HTTP のテスト）・UNIT 04（property test・fuzz・入力の読み取り）・UNIT 05（E2E・ブラウザ・アクセシビリティ）・UNIT 06（守りの回帰テスト）・UNIT 07（tracing・メトリクス・health・SLO）・UNIT 08（benchmark・profile・メモリの使い方）・UNIT 09（負荷試験・背圧・容量）・UNIT 10（CI・依存・供給網・事故の対応）は準備中で、次の更新から順に追加します。それまでの間、RustNote のテストは、`Rust/rustnote/tests/http.rs` と `tests/repository.rs`、`src/` の各ファイルの最後の `#[cfg(test)]` で読めます。

用語ミニ辞典

この巻（UNIT 01〜02）に出てきた用語です。

UNIT 01 の用語

用語	意味
ユニットテスト	関数や型のような小さな部品1つを、ほかから切り離して確かめるテスト
テストの段	ユニット・DB の結合・HTTP・E2E のように、確かめる範囲の広さで分けたテストの種類
境界値	決まりが切り替わる境目の値。端と、その1つ外側を試す
表の形のテスト（table-driven test）	入力と期待する結果の組を表に並べ、同じ確かめ方で1行ずつ試すテスト
`#[cfg(test)]`	`cargo test` のときだけコンパイルする印。本番の実行ファイルには入らない
`assert_eq!`	2つの値が等しいことを確かめる。違えば `left` と `right` を表示して panic する
わざと間違えた版	テストが間違いを見つけられるかを確かめるために、わざと壊した関数
ミューテーションテスト	わざと間違いを入れて、テストが気づくかを道具で自動に確かめる方法
ゼロ幅接合子（U+200D）	絵文字どうしをつないで1つに見せる、見えない文字
UTF-16 の単位	文字を 16 ビットずつで表したときの数。ブラウザの `maxlength` はこれで数える
`#[tokio::test]`	async のテストに付ける印。Tokio のランタイムを用意してから動かす

UNIT 02 の用語

用語	意味
結合テスト	いくつかの部品を、本物どうしでつないで確かめるテスト。ここでは、リポジトリと本物の DB
Arrange・Act・Assert	テストを、準備する・操作する・確かめるの3つに分けて書く形
偽物（fake）	テストのために作った、本物の代わりの実装。SQL は動かないので、SQL の間違いには気づけない
`common::test_db()`	テストごとに一時フォルダを作り、マイグレーション済みの空の DB を渡す RustNote の関数
`TempDir`	`tempfile` クレートの一時フォルダ。片付けられる（drop される）と、フォルダごと消える
`ESCAPE '\'`	`LIKE` の中で、`\` のうしろの文字（`%`・`_`）を、ただの文字として扱わせる指定
`VersionMismatch`	更新しようとした version が、今の version と違ったときのエラー（楽観的ロック）
共有しているもの	DB のファイル・ポート番号・環境変数・今の時刻など、テストどうしで同じものを使う所。ときどき失敗するテストの原因になる
`database is locked`	ほかの接続が書いている最中で、DB を使えなかったときの SQLite のエラー
`ON DELETE CASCADE`	参照先の行が消えたら、参照している行も一緒に消す、という表の約束